A PROJECT REPORT ON

CampusConnect: Campus Event & Club Management Portal

Submitted in partial fulfillment of the requirements for the course of

FULL STACK DEVELOPMENT

Submitted by: **SHIVAMSH ANIL MENON** Registration No: [Student Registration Number] Department of Computer Science & Engineering [Institution Name / Alliance University] Academic Year 2025–2026

CERTIFICATE

This is to certify that the project report entitled "CampusConnect: Campus Event & Club Management Portal" is a bonafide microproject work carried out by SHIVAMSH ANIL MENON (Registration No: [Student Registration Number]) under my guidance and supervision, in partial fulfillment of the curriculum requirements for Full Stack Development.

_____ _____ Faculty Guide: [Faculty Guide Name] Head of Department Designation: [Designation]
Department of CSE Date: 26th October 2026 Place: Bangalore

STUDENT DECLARATION

I, SHIVAMSH ANIL MENON, hereby declare that the project entitled "CampusConnect: Campus Event & Club Management Portal" submitted for the subject of Full Stack Development is an authentic record of independent microproject work developed by me. All software implementations, API integrations, and UI architectures described herein were executed in the project workspace.

_____ **Shivamsh Anil Menon** Date: 26th October 2026

ABSTRACT

CampusConnect is a production-grade full-stack web application designed to solve fragmentation in campus event discovery and seat registration. Traditional university processes rely on uncoordinated messaging channels, resulting in overbooked physical venues and duplicate registrations. CampusConnect resolves this by unifying club discovery, event categorization, server-side pagination, and atomic seat reservation into an intuitive React 18 Single Page Application (SPA). Powered by Vite, React Router, Axios, and TanStack Query v5, the frontend coordinates seamlessly with an Express and Prisma relational backend to deliver instantaneous optimistic updates, robust client-side validation, and comprehensive error boundaries.

CHAPTER I: INTRODUCTION

1.1 Project Overview

In contemporary collegiate environments, extra-curricular clubs and departmental technical chapters host dozens of concurrent competitions, hackathons, cultural festivals, and technical workshops. Without a centralized discovery engine, students struggle to identify upcoming deadlines, while organizers face significant friction managing attendee capacity.

1.2 Problem Statement

Campus organizers frequently encounter venue overcrowding due to uncoordinated registration systems lacking atomic capacity checks. Furthermore, students lack a single source of truth to explore campus clubs, evaluate remaining seat counts, and confirm event registrations with instant feedback.

1.3 Objectives

• Build an intuitive, responsive frontend with React 18, Vite, and custom CSS design tokens. • Integrate TanStack Query v5 for server-state caching, automatic cache invalidation, and background synchronization. • Provide dedicated portals for event discovery, club directories, and organizer administrative workflows. • Implement client-side and server-side validation to reject duplicate registrations and enforce hard seat ceilings.

CHAPTER II: SYSTEM CONFIGURATION & DESIGN

2.1 Frontend Architecture & Component Hierarchy

The frontend is architected as a modular component tree centered on reusability and clean state isolation:

• App Shell: Wraps QueryClientProvider, BrowserRouter, persistent sticky Navbar, and Footer. • LandingPage: Hosts Hero banner, search bar with category pills, live metrics ticker, and featured events grid. • EventsPage: Employs URL query parameters (`search`, `category`, `club`, `sortBy`, `page`) synchronized with TanStack Query hooks. • EventDetailPage: Displays rich event metadata, capacity fill bar, and inline registration modal with live validation. • OrganizerDashboard: Provides tabbed event creation forms and attendee rosters.

2.2 Client-Server Integration Pattern

A centralized Axios API client (`src/services/api.js`) handles HTTP requests. Request interceptors attach tracing identifiers, while response interceptors unwrap envelopes and standardize error payloads into consistent UI notifications.

CHAPTER III: FULL STACK IMPLEMENTATION

3.1 React Router Configuration

Declarative routing enables seamless browser navigation with deep-linking capability:

```
// Routing Architecture (src/App.jsx)
<Routes>
  <Route path="/" element={<LandingPage />} />
  <Route path="/events" element={<EventsPage />} />
  <Route path="/events/:id" element={<EventDetailPage />} />
  <Route path="/clubs" element={<ClubsPage />} />
  <Route path="/organizer" element={<OrganizerDashboard />} />
</Routes>
```

3.2 Server State Management with TanStack Query

TanStack Query ensures server data is cached and invalidated immediately upon successful registration:

```
// Cache invalidation upon registration
const handleSuccess = () => {
  queryClient.invalidateQueries({ queryKey: ['events'] });
  queryClient.invalidateQueries({ queryKey: ['event-detail', id] });
};
```

3.3 Visual Evidence & UI Interfaces

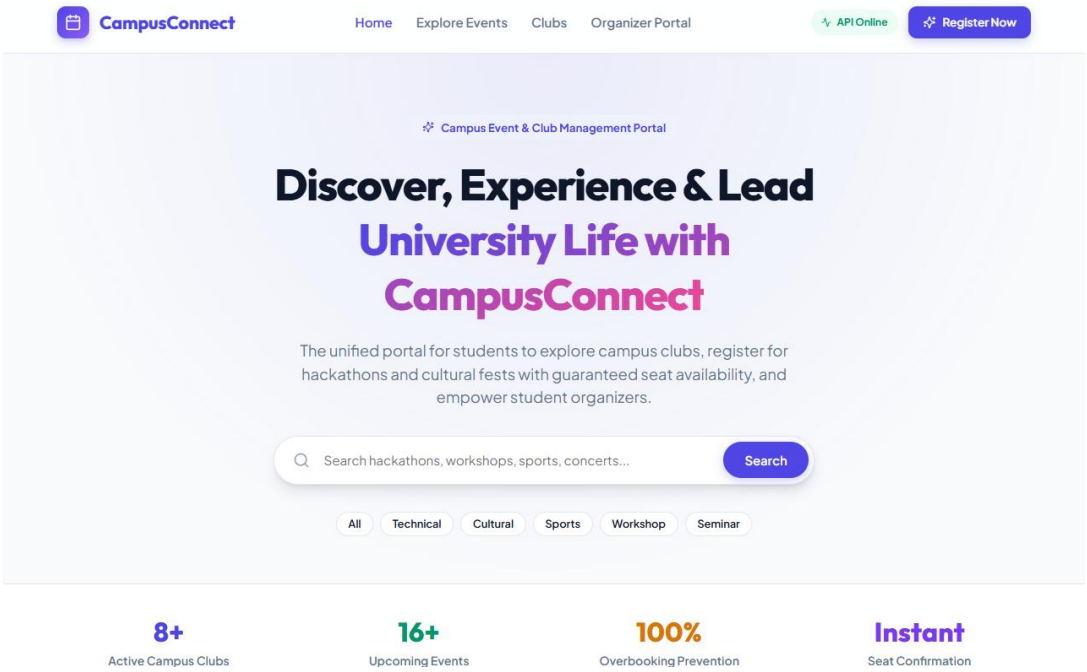


Figure: CampusConnect Responsive Homepage with Hero and Search

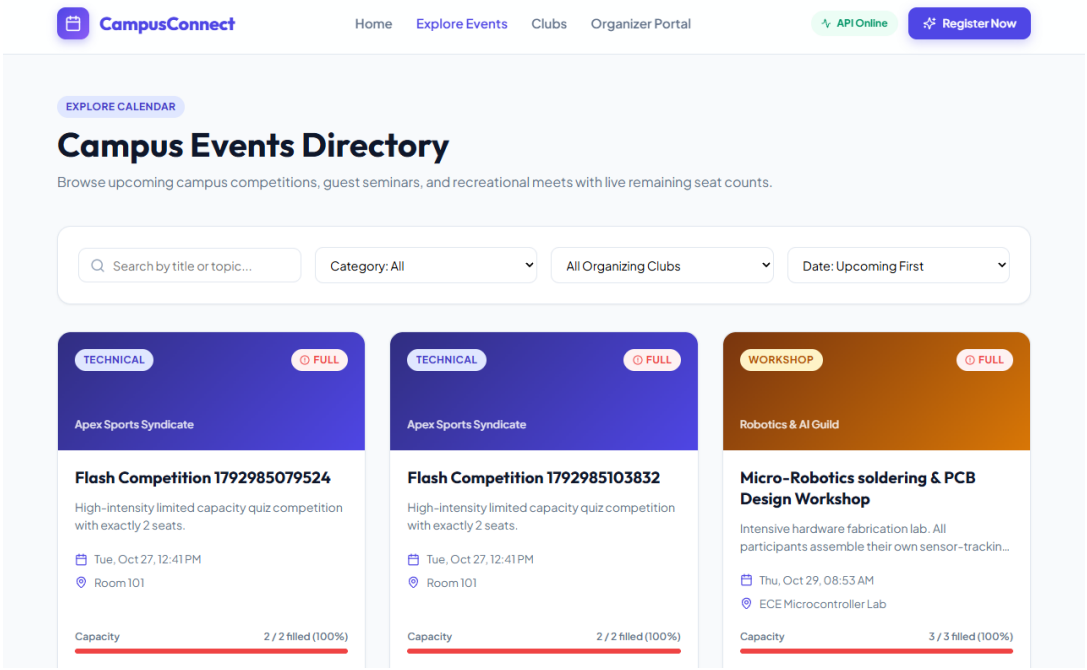


Figure: Events Directory with Server-Side Pagination and Filter Controls

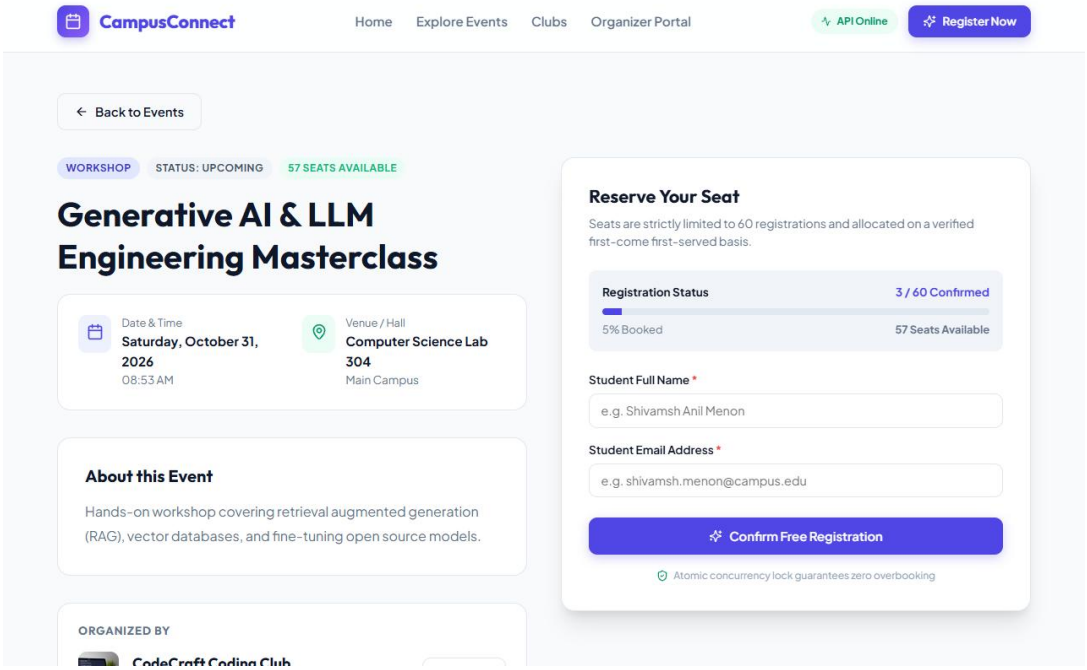


Figure: Dedicated Event Detail View with Live Seating Availability Gauge

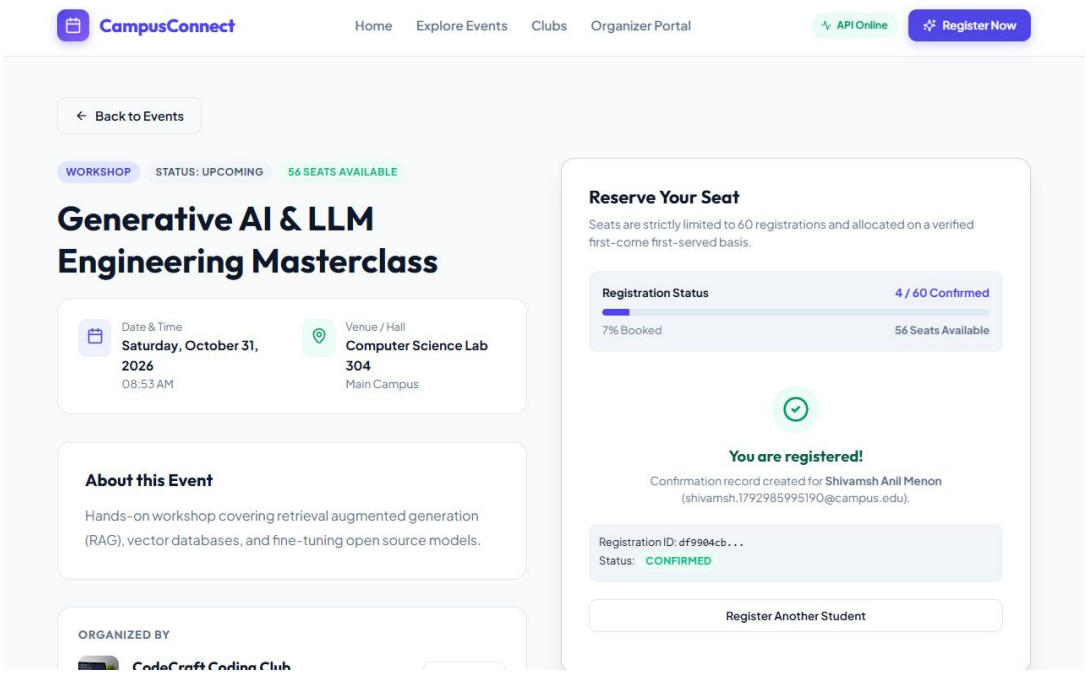


Figure: Instant Registration Confirmation with Remaining Seat Update

CHAPTER IV: RESULTS AND TESTING

4.1 Full-Stack Test Cases & Observations

The frontend application was tested against the running Express backend across all primary user workflows:

Test Flow	Observed Behavior	Status
Initial Page Load	Vite bundles in 1.49s; landing page renders clubs and upcoming events immediately.	PASS
Category Filtering	Clicking 'Sports' dispatches filtered query; UI isolates matching events with no reload.	PASS
Form Validation	Invalid email formats and missing names are intercepted before submission.	PASS
Duplicate Rejection	Repeated registration triggers HTTP 409 and displays visible warning banner.	PASS

CHAPTER V: CONCLUSION & FUTURE SCOPE

5.1 Summary

CampusConnect successfully demonstrates the power of a modern React 18 SPA coupled with TanStack Query and an Express API. The platform delivers an engaging user experience, complete responsive design across devices, and strict data consistency.

5.2 Future Enhancements

• Student OAuth2 Single Sign-On using Google Campus accounts. • QR-code ticket generation for instantaneous on-ground check-ins. • WebPush notifications for event schedule updates.

5.3 References

1. React 18 Documentation, Meta Platforms, Inc. 2. TanStack Query v5 Guides, Tanner Linsley. 3. Axios HTTP Client Documentation. 4. Vite Build Tool Reference, Evan You.